

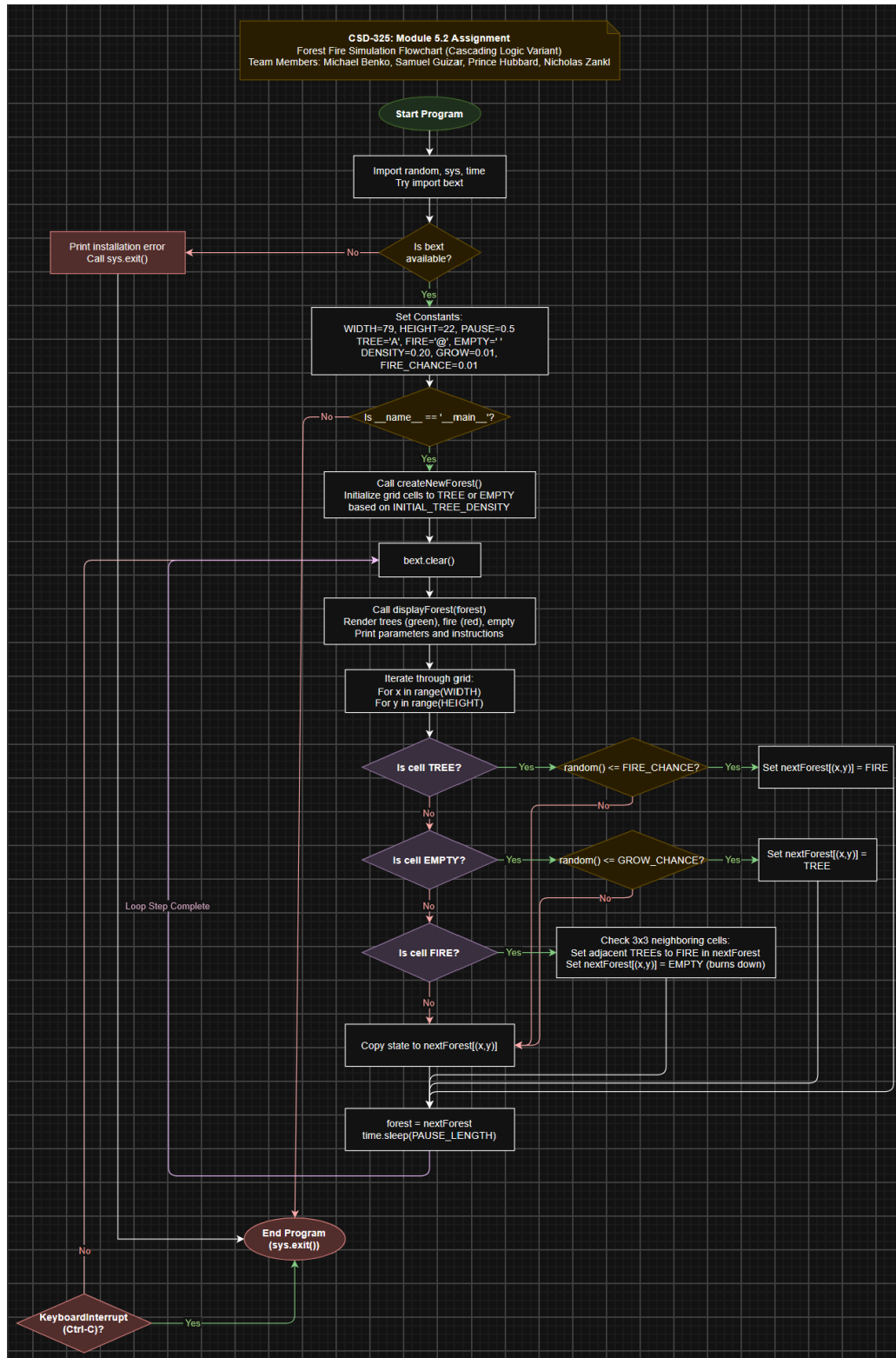
## CSD-325: Module 5.2 Assignment

### Forest Fire Simulation Flowchart

Date: September 6, 2026

Group C Members: Michael Benko, Samuel Guizar, Prince Hubbard, Nicholas Zankl

Figure 1: System Execution Lifecycle and Cascading Pixel Logic for forestfiresim.py



## Program Logic and Flowchart Mapping Analysis

The flowchart for `forestfiresim.py` maps the full execution lifecycle of the simulation, tracing every step from initial environment setup to real-time grid evaluation and process termination. The diagram aligns directly with the script's architectural layers in a continuous, logical progression.

### Environment Setup & Dependency Verification

Execution begins at the **Start Program** oval and proceeds into the environment setup phase. The script attempts to import required libraries including `random`, `sys`, `time`, and `bext`. The **Is bext available?** decision diamond maps the try-except `ImportError` block. If `bext` is missing, the diagram follows the **No** path to display installation instructions and trigger `sys.exit()`. If `bext` is verified, the diagram follows the **Yes** path to the **Set Constants** block, where grid parameters (`WIDTH`, `HEIGHT`), state symbols (`TREE`, `FIRE`, `EMPTY`), and probabilities (`INITIAL_TREE_DENSITY`, `GROW_CHANCE`, `FIRE_CHANCE`) are instantiated. Immediately following constant initialization, the **Is \_\_name\_\_ == '\_\_main\_\_'?** decision node acts as Python's standard module guard check. Executing the script directly routes down the **Yes** branch into `main()`, whereas importing the script as a library routes down the **No** branch directly to the **End Program** node to prevent accidental execution.

### Initialization & Screen Setup

Moving into `main()`, the process box **Call createNewForest()** represents the function that populates the initial two-dimensional coordinate dictionary. Each cell evaluates a random probability roll against `INITIAL_TREE_DENSITY` (20%) to assign either a **TREE** ('A') or **EMPTY** (' ') state. Once populated, the **bext.clear()** process box executes a terminal screen wipe, setting up a clean canvas before entering the primary double-buffered evaluation loop.

### Live Simulation & Cascading Evaluation Loop

The main simulation loop opens at **Call displayForest(forest)**, which repositions the terminal cursor to (0, 0) via `bext.goto(0, 0)` and renders the current frame character-by-character using green for trees and red for fires. Following display, the nested coordinate loop (**For x, For y**) evaluates every pixel sequentially. To model Python's if-elif-else control structure cleanly without overlapping paths, the decision diamonds are arranged in a cascading vertical sequence:

First, **Is cell TREE?** inspects whether the active coordinate contains 'A'. On a **Yes** result, execution moves right to evaluate `random() <= FIRE_CHANCE` (1% chance of a lightning strike). A successful roll sets `nextForest[(x,y)] = FIRE`, whereas an unsuccessful roll routes into the default state copy block. If the cell is not a tree, the **No** path falls straight down into **Is cell EMPTY?** On a **Yes** result, execution moves right to evaluate `random() <= GROW_CHANCE` (1% growth chance). A successful roll sets `nextForest[(x,y)] = TREE`, while an unsuccessful roll routes to the default state copy block. If the cell is neither a tree nor empty, the **No** path falls straight down to **Is cell FIRE?** On a **Yes** result, execution triggers the neighbor inspection process, checking all eight surrounding coordinates (`x + ix, y + iy`). Any adjacent **TREE** in forest is marked as **FIRE** in `nextForest`, and the original burning tree transitions to **EMPTY**. Finally, if a cell evaluates to **No** across all condition checks, execution drops straight down into **Copy state to nextForest[(x,y)]**, preserving the cell's existing state.

## Buffer Swap & Program Exit

Once all grid coordinates finish evaluation, execution enters the **forest = nextForest / time.sleep(PAUSE\_LENGTH)** process box. Live states are updated simultaneously across the double-buffered frame, and execution pauses for 0.5 seconds. The **Loop Step Complete** connector routes from the swap block back up to **bext.clear()**, closing the frame cycle and starting the next render loop. This loop continues indefinitely until the user presses **Ctrl-C**, where the red **KeyboardInterrupt (Ctrl-C)?** decision diamond catches the signal and routes execution cleanly to **End Program (sys.exit())**.